The Journal of Supercomputing (2022) 78:16056–16087
https://doi.org/10.1007/s11227-022-04520-5

An efficient mutual authentication scheme for IoT systems

Islam Alshawish[1] · **Ali Al-Haj**[1]

Accepted: 7 April 2022 / Published online: 30 April 2022
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

The resource-constrained nature of Internet of Things (IoT) devices and their diversity and abundancy represent a major challenge for the development of efficient security solutions that fit IoT systems and ensure end-to-end security. This paper presents an efficient IoT authentication scheme that provides mutual authentication between three different entities making up the IoT system under study; IoT devices, an IoT manufacturer server, and an authentication server. The authentication process is accomplished by the exchange of only six small messages in three successive stages: the IoT device association stage, the IoT device–AS server communication initialization stage, and the exchanged data encryption stage. The scheme employs the HMAC keyed hash function to provide mutual authentication and the AES-GCM lightweight symmetric cryptographic function to provide integrity and confidentiality for the exchanged data. The proposed scheme was simulated using the Network Simulator NS3 with C++ crypto++ library, using the various IoT enabling technology models. The obtained simulation results verified the proper functioning of the designed scheme and demonstrated the efficiency of the proposed scheme in terms of the low computational and communication overheads. The scheme needed 13.04 ms and 470 bytes to complete the mutual authentication process, including session key exchange. The low computational and communicational overhead of the proposed scheme are due to the scheme's small-sized messages, lightweight cryptographic functions, and passive discovery of IoT device. Security analysis of the scheme shows that the essential security requirements, which includes mutual authentication, confidentiality, and integrity, have been met by the proposed authentication scheme. Moreover, the scheme is inherently robust against common cyber-attacks such as the DoS, impersonation, spoofing, sniffing, and man-in-the-middle attacks.

Keywords Internet of Things (IoT) · Mutual authentication · Lightweight cryptography · Network simulation

✉ Ali Al-Haj
ali@psut.edu.jo

Extended author information available on the last page of the article

1 Introduction

The rapid growth and distribution of IoT systems drive many security and privacy concerns that have to be addressed and solved, to ensure data safety from cyber-attacks and unauthorized access [1–4]. However, addressing such concerns is challenged by the diversity of the IoT devices which results in differentiation of the security requirements. That is, while some applications demand a high level of security, such as ensuring data confidentiality in addition to its integrity, others focus on a minimum level like data integrity only. An important requirement for IoT systems consisting of many nodes is to mitigate the influence and the impact of any compromised or malicious nodes on the rest of the system. This can be done by ensuring identity validation and authentication of these nodes before granting them access to the network [1, 5–7].

Implementing mutual authentication among the different entities of distributed and scalable IoT systems is a fundamental requirement. These entities include IoT devices, IoT gateways and middleware servers, cloud servers, and remote user/applications. As shown in Fig. 1, three levels of authentication among these entities can be defined: level A authentication between an IoT device and a gateway or middleware service, level B authentication between the IoT devices themselves, and level C authentication between an IoT Cloud service and a user/application. End-to-end authentication is achieved through all authentication levels.

The IoT systems are resource-constrained devices connected over low-power and lossy networks [3, 8], and thus, the traditional mutual authentication solutions cannot be used since they rely heavily on public key cryptography [1, 6, 9–12]. The computation-intensive public key cryptography imposes a heavy computational burden on the resource-constrained IoT devices and consume large amounts of time and power for the transmission of large public key certificates [5]. Hence, comes the need for lightweight and efficient solutions that cope with the inherent IoT characteristics and support the scalability and distribution of IoT systems to ensure achieving the essential security requirements; confidentiality, integrity, availability, and authenticity [13, 14]. It is indeed major a requirement that efficient security solutions

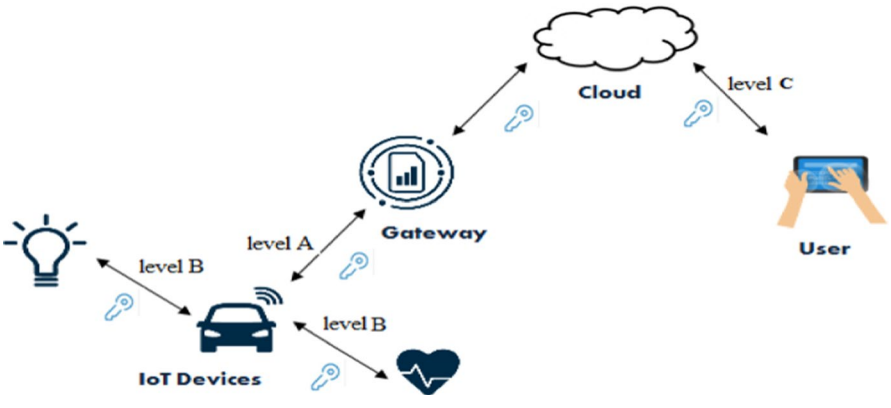


Fig. 1 IoT mutual authentication levels

perform lightweight computation on the IoT resource-constrained devices, provide time and power saving, and generate minimal communicational and computational overheads [1, 9, 15–17].

There are many papers in the literature that have been dedicated to IoT security issues and challenges. These papers review the efficiency of proposed algorithms according to their security, power, time, and applicability to the IoT devices and systems [18–28]. As IoT authentication is a major issue in any IoT security solution, the authors in [16, 29] presented a comprehensive surveys on IoT authentication protocols. They concluded that improved authentication protocols for IoT systems are still needed. For this purpose, many IoT mutual authentication schemes have been proposed in the literature [4, 7, 9, 17, 30–37]. However, most of these schemes have a major limitation which is overlooking the inherent resource-constrained characteristic of IoT devices. The schemes reported in [30, 36, 37] impose extensive computational and communicational overhead on the IoT devices, while schemes [31, 33–35] fail to guarantee the essential security requirements which includes confidentiality and authenticity of exchanged messages between communicating entities.

In this paper, a new lightweight, two-factor mutual IoT authentication scheme is proposed. The scheme establishes mutual authentication between the resource-constrained IoT devices and a middleware authentication server. The authentication process is accomplished by the exchange of six small messages, thus avoiding high computational and communication overheads. The scheme deploys secure-by-design principle by involving the IoT manufacturer as a major player in the authentication process in order to achieve automatic authentication for scalable and distributed IoT systems. The scheme validates identity of the communicating parties as a first factor for authentication and uses a shared secret key as the second factor. This two-factor authentication assures the communicating entities that they are communicating with each other and not with intruders. The scheme also performs a session key agreement to secure any further communications. For cryptographic operations, the proposed scheme employs the HMAC keyed-hash function to provide mutual authentication [38], and the AES-GCM lightweight symmetric cryptographic function [39–41] to provide integrity and confidentiality for the exchanged data. The cryptographic keys establishment process includes; session key generation, transportation, agreement and confirmation in a fully automation way without any sort of human intervention. This automatic handling of the pre-shared key establishment process provides dynamicity and scalability to the IoT system, and it prevents human errors.

The proposed scheme was simulated using the Network Simulator NS3 with C++ crypto++ library, using the various IoT enabling technology models like such as Wi-fi, 6LoWPAN, and IEEE 802.15.4-based LR-WPAN. The obtained simulation results verified the proper functioning of the designed scheme and demonstrated the efficiency of the proposed scheme in terms of the low computational and communication overheads. The scheme needed 13.04 ms and 470 bytes to complete the mutual authentication process, including session key exchange. The low computational and communicational overhead of the proposed scheme are due to the scheme's small-sized messages, lightweight cryptographic functions, and passive discovery of IoT device. Security analysis of the scheme shows that the essential security requirements, which includes mutual authentication, confidentiality, and

integrity, have been met by the proposed scheme. Moreover, the scheme is inherently robust against common cyber-attacks such as the DoS, impersonation, spoofing, sniffing, and man-in-the-middle attacks [42].

The rest of this paper is organized as follows. Section two reviews recent IoT authentication schemes, highlights the research gaps, and clarifies the contributions of this paper. This is followed by a detailed description of our proposed IoT authentication scheme in section three. The performance evaluation results of the proposed scheme are presented and analyzed in section four with detailed comparison with relevant schemes. Section five concludes this paper and presents future work directions.

2 Literature review

A fundamental requirement in IoT security is to implement mutual authentication among the different entities of IoT systems. These entities include IoT devices, IoT gateways and middleware servers, cloud servers, and remote user/applications. In this section, we present review of a number of recently proposed IoT authentication protocols. The review aims to explore the state-of-the-art solutions in this advancing field, identify their limitations and gaps, and outline the solutions set by the proposed scheme to overcome the identified gaps and limitations.

2.1 IoT mutual authentication schemes

The authors in [30] proposed a certificate-based secure communication scheme for achieving mutual authenticity between the communicating entities of IoT systems. The proposed scheme is based on using ECC public key cryptography to design cryptographic certificates for the IoT devices. Additionally, the scheme uses digital signatures stored alongside the certificates on each IoT device. Although the proposed scheme was intended to be lightweight in terms of computational and communication overheads, the scheme relied heavily on public key cryptography which is computation-intensive. That is, the two ECC keys used by the scheme requires longer processing times compared to symmetric keys for the same level of security. Furthermore, the proposed scheme required 8 messages of total 1639 bytes indicating relatively high computational and communicational overheads.

The schemes presented in [43, 44] achieve secure transmission of data between IoT networks and remote users by ensuring mutual authentication and session key negotiation between the different entities of the IoT system. In these schemes, mutual authentication is achieved using an ECC-based three-factor remote user authentication. An obvious limitation of these schemes is that ECC-based scheme requires extensive usage of resources leading to relatively high communication and computation overheads.

The authors in [31] developed a keying protocol that establishes a membership credential for the IoT devices that are newly added to a network. The protocol is based on using a pre-shared symmetric key between a resource-constrained

IoT device, called a child, and a resource-rich device, called a guardian. A pre-established trust relationship is assumed to exist between these two devices and a third-party device called the parent. The parent devices provides the child and the guardian devices with symmetric keys to be used for secure inter-device communication. Although the proposed protocol uses lightweight symmetric cryptographic functions to suit the resource-constrained IoT devices, the protocol does not check the validity of the claimed identity of the IoT devices upon their request to join the network. This is a major drawback of the protocol since validating the identity of the IoT devices is indeed the first and most important issue in IoT authentication.

In [32], an inter-device authentication and a session key distribution scheme is proposed. In the context of increased demands on lightweight security solutions that fit well for resource-constrained IoT devices, the implementation of the proposed scheme has been based on a single hash/cipher cryptographic primitive. However, the scheme relies on the resource-constrained IoT devices for the generation of the session keys, instead of relying on a centralized key distribution center. This a major drawback of the scheme since the IoT devices would definitely need their constrained resources to perform their original tasks (sensing, collecting data, actuating, among other tasks).

The scheme proposed in [33] describes a mutual authentication scheme between an IoT device and a personal area network coordinator (CPAN). The coordinator has sufficient processing resources in order to authenticate any IoT device. Upon successful mutual authentication, the scheme establishes a secure communication channel between the authenticated parties. A major drawback of this scheme is the manual installation of pre-shared keys in the communicating parties. This is prone to human-caused faults and limits scalability of the system.

A mutual authentication protocol between a resource-constrained sensor node and a remote end user, is proposed in [34]. The scheme is based on a certificate-less public key cryptography. As many mutual authentication schemes, the ultimate goal of this scheme is to ensure secure sharing of a cryptographic key between the communicating entities. To achieve this goal, the proposed scheme relies on an authenticated coordinator node to establish a secure channel between an IoT node and an end user. Although this scheme is about mutual authentication and session key establishment between a sensor node and an end user, the scheme does not provide any information on how this authentication is done.

The scheme proposed in [35] provides mutual authentication between an IoT device and a gateway over a wireless communication channel. The protocol is based on the symmetric cryptographic algorithm AES-CCM and the HMAC-based key derivation function. Moreover, the protocol relies on prior establishment of a shared key using a master key and a session key with an initial value. A unique master and session keys pair is installed manually into each IoT device and in the gateway. This scheme lacks scalability due to the fact that the secret credentials are installed manually by the administrator of the network. Another drawback is the involvement of the source-constrained IoT devices in computing the shared session key, which means adding additional burden to such resource-constrained devices.

The scheme proposed in [36] describes an IoT authentication and key management scheme based on signature schemes. The authors adopted the system model where all IoT devices are connected to the Internet through a gateway. The IoT devices are accessed by a system user through this gateway after establishing mutual authentication between the user and the gateway. The scheme provides an end-to-end authentication since it provides authentication and secure communication between users/applications and the IoT devices, through the middleware gateways. This communication takes place only if both the application and the IoT device can authenticate each other. The proposed scheme employs elliptic curve cryptography digital signature schemes.

The authors of the IoT authentication scheme proposed in [37] make use of the Datagram Transport Layer Security protocol (DTLS) as their end-to-end security architecture. The scheme employs public key cryptography, TLS-RSA-with-AES-128-CBC-SHA, to provide mutual authentication between clients and servers. Any two communicating parties must negotiate issues such as the keys and cipher algorithms to use before any transfer of data takes place. Although this scheme is considered to be robust, its major drawback is using the computation-intensive public key cryptography.

2.2 Limitations of surveyed schemes and proposed solutions

Referring to the literature survey described so far, the limitations of the reported IoT authentication schemes have been identified and are listed in Table 1. As given in the table, most of the proposed schemes suffered from large amounts of undesirable communication and computation overheads. This is an indication of the failure of these schemes to provide the needed robust authentication while taking into consideration the fact that the IoT device is resource-constrained devices.

Our proposed scheme overcomes the identified limitations as described in Table 1. The specific solutions presented by the proposed scheme represents the overall contribution of this paper with regard to providing effective and efficient mutual authentication for IoT systems. The overall contribution of this paper can thus be summarized as follows:

- Developing a lightweight two-factor mutual authentication and key exchange and agreement scheme.
- Maintaining the constrained resources of the IoT devices by presenting an efficient scheme with minimal communication and computation overheads as well as with best time efficiency.
- Applying an innovative trust model based on IoT device, IoT manufacturer, and a middleware relations.
- Providing scalability to the proposed IoT authentication scheme by automating the key distribution process and eliminating human intervention.
- Using the state-of-the-art NS3 simulator to evaluate the scheme's impact on communication, network and time performance parameters.

Table 1 Limitations of relevant IoT mutual authentication schemes and our proposed solutions

	Drawbacks/limitations	Proposed scheme's solution
[30]	Using two elliptic keys with large encryption keys	Deployed lightweight cryptography with much smaller symmetric keys
	Relatively high communication and computation overheads and longer time	Minimal computation and communication overheads
	Scheme implemented by 8 messages of total 1639 bytes	Scheme implemented with 6 messages having an overall size of 470 bytes
[31]	Checking the validity of the claimed identity of the IoT devices upon their request to join the network is not supported. This is a major drawback of the protocol since validating the IoT identity is the first and most important issue in IoT authentication to prevent malicious entities intrusion	Checking the validity of the claimed identity of the IoT devices is done through provisioning of a secret key
	Four messages are sent in plaintext, which not only increases the possibility of attacks but also leads to consumption of more IoT resources in case of resending these messages when an attack is launched	Exchanged messages are secured through encryption
[32]	Relies on the resource-constrained IoT devices for session keys generation; thus, it poses IoT devices to extra computational overhead	Relies on resource-rich entity for session keys generation, thus saving IoT energy with less computational overhead
	Based on the manual process of installing secret keys by the network administrator, thus limiting scalability of the IoT network	Pre-shared key is saved on IoT at manufacturing time and handed over to the middleware server with no human intervention
	Assumes the existence of a secured communication channel for the IoT devices, and not establish one	Establishment of a secured channel using three clear and successive stages
[33]	Installing the secret keys manually by the network administrator which prone to human-caused faults and limits scalability of the system	Human intervention is eliminated
	Sending secret credentials in plain text, thus increasing the risk of sniffing and other attacks	Encryption of transmitted data
[34]	The authors do not specify how the secured channel is established	Establishment of a secured channel using three clear and successive stages
	It issues many "hello messages" to the IoT device and the device has to maintain a lot of "coordination requests" in order to get the service	Passive node discovery: IoT initiates association request
	Suffers from extensive communication overhead; at least 9 messages to be exchanged	Reducing number of exchanged messages to 6 messages only, thus minimizing communication overhead. This has been a preset design objective
[35]	- Lacks scalability as the secret credentials are installed manually by the administrator of the network	Eliminating human intervention and offloading additional computations to a the AS server which has higher computational capabilities

Table 1 (continued)

	Drawbacks/limitations	Proposed scheme's solution
	Involves the source-constrained IoT in computing the session keys, thus adding additional computation overhead to the resource-constrained IoT devices	
[36, 37, 43, 44]	Using public key cryptography poses computational and communicational overheads to IoT devices	Using lightweight cryptography with minimal computational and communicational overheads

- Presenting detailed security analysis of the scheme to prove that the scheme is secure and meets the essential security requirements, and is indeed robust against most common attacks on IoT systems.

3 The proposed mutual authentication scheme

For an IoT device to join its IoT network in a secured manner, it has to have a trust relationship with the network's entities. However, to establish such a trust relationship, the device must first get the necessary credentials. As IoT end devices are classified as resource-constrained devices, they cannot perform the different security tasks without relying on authentication servers with rich computing resources. However, some sort of device–server authentication is essential since it is here where the IoT device is able to join the network and to get authenticated to carry out its preassigned tasks. The proposed scheme aims to design and implement an authentication and key exchange protocol for IoT devices and systems. This protocol addresses the issue of mutual authentication between the IoT entities and the design of the appropriate exchanged messages between these entities. In this section, the IoT entities will be described and the authentication protocol among these entities will be presented by giving details of the exchanged messages.

3.1 The IoT entities

The proposed scheme aims to achieve the fundamental and basic level of authentication in IoT systems. This basic level, named level A in Fig. 1, is concerned with providing mutual authentication between the IoT devices and an authentication server. Hence, the involved entities are as follows: IoT devices, IoT Manufacturer Server (Man.S), and Authentication Server (AS). No additional and specific hardware or special entities are required. The AS server could be the IoT gateway itself and Man.S is the producer of the IoT device. Following is a brief description of each involved entities and its role in the proposed scheme.

IoT devices. An IoT device is the "*thing*" that does certain tasks, gains, and shares data to other "*thing*" entities or to more powerful devices, users, and applications. The IoT devices are uniquely identified and connected to the Internet. An IoT device has some amount of nonvolatile storage or memory.

IoT manufacturer server (Man.S) The role of the manufacturer server in the proposed authentication scheme goes beyond the secure updates for the IoT software. It is supposed to be an integral part of the authentication scheme, which eventually grants the IoT device access to the network and the Internet. The tasks that the IoT manufacturer server has to do are as follows: supply the IoT device with the prestored identity credentials, have a list of trusted authentication servers to which the IoT device can be associated with in order to join the network, and confirm the identities of its manufactured IoT devices when they request the authentication server for network association.

Authentication server (AS) The authentication server is considered as an edge computing servicing entity that resides between the ground IoT devices and the cloud servers or remote applications. Any IoT device needs to contact AS for authentication in the first place. After being authenticated, the IoT device is able to perform its pre-assigned tasks or establish connections with other entities in the IoT network. The authentication process is done based on an assumed trust relationship between the Man.S and AS servers. Here the secret key stored in the IoT device by the manufacturer is used as a pre-shared key between the IoT device and the AS server in order to set up the mutual authentication process. The trust relationship between the AS and Man.S servers is important in order to share the IoT key to the AS server in secured manner. This key is also used by the IoT device to perform encryption, decryption, and HMAC operations.

3.2 Details of the proposed scheme

In this section, the proposed scheme is described in detail. The detailed description of the scheme starts with stating the assumptions and initializations on which the scheme has been based. This is followed by presenting the overall operational design flow of the scheme. Finally, detailed formats of the exchanged mutual authentication messages will be described. An simple architectural view of the proposed scheme is shown in Fig. 2.

3.2.1 Assumptions and initializations

Similar to any authorization and authentication schemes, a set of assumptions and initializations on which the scheme is based are clearly stated. The assumptions are as follows:

- All the entities are physically protected against any physical attacks.
- Man.S and AS server have a pre-established trust relationship.

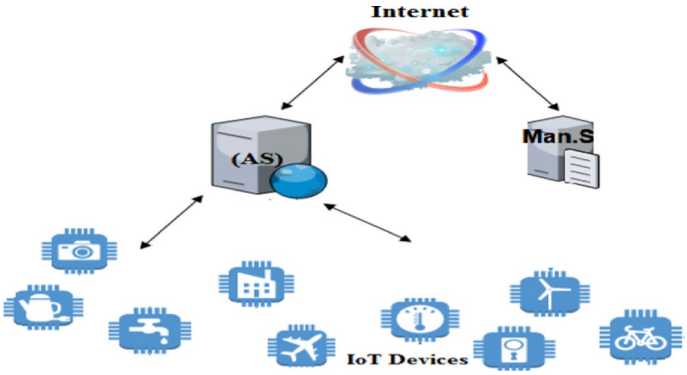


Fig. 2 Overall proposed system architecture

- Each IoT device has pre-stored secret key K_{IoT} by provided by its manufacturer Man.S
- The Man.S server has two lists: L_{IoT} which lists the IDs of IoT devices and their stored keys (K_{IoT}), and the L_{AS} which lists the trusted AS severs.
- A backup AS server is configured to maintain availability and to avoid single point of failure which could raise due to sudden failure of the AS server.
- Discovery of the IoT devices is done passively. That is, when an IoT device wants to join the network, it initiates a registration request.
- The secret key K_{IoT} is used as a pre-shared key between the IoT device and the AS server to set up the mutual authentication process.
- The secret key K_{IoT} is delivered to the AS server by the Man.S server.
- The AS and Man.S servers are high performance machines with rich computing and communication resources. Therefore, the two servers are able to perform public key cryptography without affecting the IoT devices or the overall performance of the scheme.
- The attacker is a software adversary only.

3.2.2 Operational description and design flow

(i) A newly launched IoT device that wishes to join the IoT network sends a joining request to a pre-assigned AS server for authentication. To avoid single point of failure which could be raise from the unavailability of this AS server, one can configure a backup AS server that IoT could communicate with. This step can be expressed as shown in Eq. (1).

$$\text{IoT} \rightarrow \text{AS} : \left\{\text{ID}_{\text{IoT}}, \text{ID}_{\text{Man.s}}\right\} \tag{1}$$

(ii) The AS server is responsible for accepting or denying this request-to-join message based on two authentication factors. The first factor is the IoT Identity (ID_{IoT}) itself, which is provided through the trust relationship between the Man.S server of the IoT device and the requested AS server. That is, it is pre-assumed that the IoT Man.S server has IDs for all trusted IoT devices. These device–ID pairs have been set by the manufacture of the IoT devices at the time of their production. This ID verification is expressed in Eq. (2), where Pbk_{AS} refers to the public key for AS server.

$$\text{AS} \rightarrow \text{Man.S} : \left\{\text{ID}_{\text{IoT}}, \text{ID}_{\text{AS}}, \text{Pbk}_{\text{AS}}\right\}, \quad \text{where} \quad \text{Pbk}_{\text{AS}} \tag{2}$$

Then the Man.S server can validate the identity of the IoT device using the ID provisioned to the AS server from the IoT device. In other words, the Man.S server checks validity of AS server and IoT IDs in its trusted lists. This is expressed in Eq. (3) where L_{AS} is the list of trusted AS severs at Man.S and L_{IoT} is the list of the IDs of IoT devices at Man.S.

$$\forall \text{ID}_{\text{AS}} \in L_{\text{AS}} \,\&\, \text{ID}_{\text{IoT}} \in L_{\text{IoT}} \tag{3}$$

The second authentication factor is the pre-shared key stored in the IoT device K_{IoT}. The AS server and the IoT device use this key as a pre-shared key to set up the mutual authentication process. This key is provisioned to the AS server from the Man.S server if the IDs are verified. Man.S sends back to the AS server the verification of $\mathrm{ID}_{\mathrm{IoT}}$ with its K_{IoT} encrypted with $\mathrm{Pbk}_{\mathrm{AS}}$. This operation is expressed in Eq. (4), where K_{IoT} is the stored key of the IoT device.

$$\mathrm{Man.S} \Rightarrow \mathrm{AS} \left\{ \mathrm{K_{IoT}} \right\} \tag{4}$$

(iii) Once the AS server and the IoT device authenticate each other, after this challenge–response process which aimed to proof the ownership of the secret stored key K_{IoT}, the AS server generates a new session key *Ks* using the secret credentials and nonce numbers. This operation expressed in Eq. (5) where *Ks* refers to session key generated by AS.

$$\begin{gathered} \mathrm{AS} \Rightarrow \mathrm{IoT} : \{challenge\} \\ \mathrm{IoT} \Rightarrow \mathrm{AS} : \{response\} \\ f(\mathrm{sec}\, ret\, key,\ nonce) = Ks \end{gathered} \tag{5}$$

This *Ks* key is used to provide security for the data exchanged between the IoT device and the AS server. The keys and the cryptographic hash functions are exchanged between the IoT device and the AS server before the application data are transferred securely by the AES-GCM lightweight symmetric cryptographic function [45]. This is expressed in Eqs. (6).

$$\mathrm{AS} \Rightarrow \mathrm{IoT} : \{Ks\} \tag{6}$$

(iv) The IoT device can utilize this mutual authentication with the AS server to set up further authentication relationships with other entities that the AS server is associated with. This is considered as an end-to-end authentication and will be dealt with as a future work.

3.3 Exchanged mutual authentication messages

The IoT-AS Authentication protocol is executed by exchanging six messages, grouped in three main stages: the IoT device association stage, the IoT device and AS server communication initialization stage, and the exchanged data encryption stage. In what follows, the details of each stage, including the exchanged messages, are presented. In addition to the detailed description, which is also illustrated in Figs. 3, 4, 5, 6, 7 and 8, a compact illustration of the whole scheme, with all authentication messages included, is shown in Fig. 9.

3.3.1 The IoT device association stage

The aim of this stage is to establish the mutual authentication between an incoming IoT device and an associated AS server. Three entities interact in such a way to provide mutual authentication. The three entities are the IoT device, the AS server, and



Fig. 3 Message one: IoT device to AS server association request

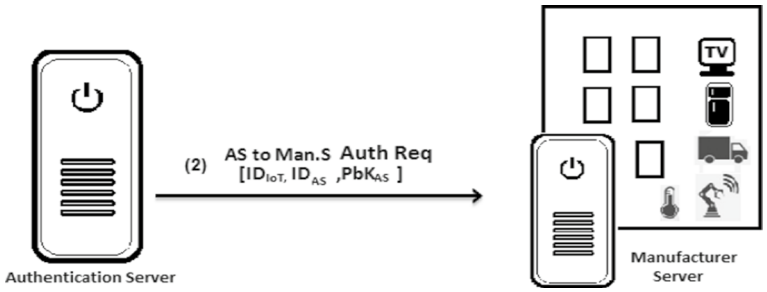


Fig. 4 Message two: AS server to IoT manufacturer server authentication request

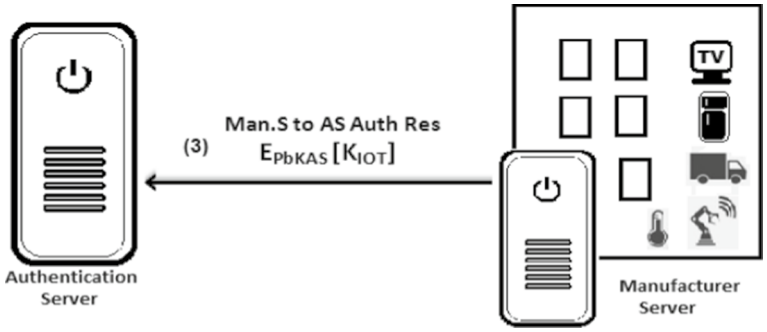


Fig. 5 Message three: IoT Manufacturer to AS server authentication response

the Man.S server. The interaction details between the three entities are shown in the following three messages.

Message 1: Association request from IoT device to AS server

Once an IoT device wants to join the network for the first time, it communicates with a pre-assigned AS server by sending an Association Request (*Assoc_Req*) which includes its identity ID_{IoT} and its manufacturer's identity $ID_{Man.S}$. The statically pre-assigned AS server in the IoT device will receive this request. This message is illustrated in Fig. 3.

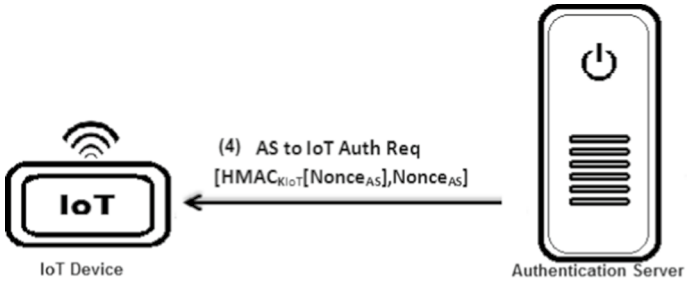


Fig. 6 Message four: AS server to IoT device authentication request

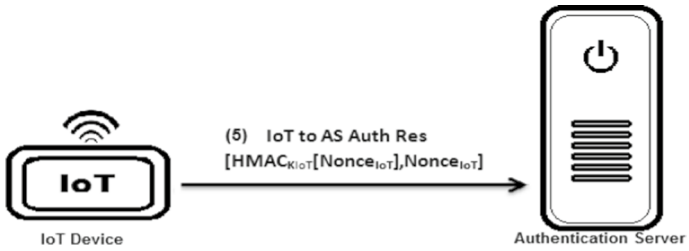


Fig. 7 Message five: IoT device to AS server authentication response

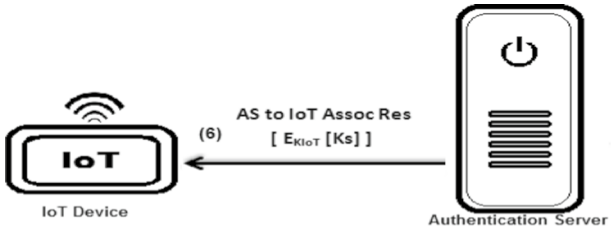


Fig. 8 Message six: AS server to IoT association response

Message 2: Authentication request from the AS server to the IoT manufacturer

Once the AS server receives the (*Assoc_Req*) from the IoT device, it sends an Authentication Request to the manufacturer of the IoT device (*AS to Man.S Auth Req*). The aim of this request is to validate the IoT device identity, ID_{IoT}, and to verify that it truly belongs to this manufacturer. This request includes identification parameters of the IoT device ID_{IoT}, the identity of the AS server ID_{AS}, and the public key of the AS server PbK_{AS}. This message is illustrated in Fig. 4.

Message 3: Authentication Response from IoT Man.S server to the AS server

The IoT *Man.S* server checks existence of the ID_{IoT} and ID_{AS} identities in a pre-stored trust list. If a match is found for both identities, then the Man.S server sends back an Authentication Response message to the AS server (*Man.S to AS Auth Res*). The response message contains the securely saved key at the IoT device, K_{IoT}, encrypted using the AS server's public key PbK_{AS} which the IoT Man.S server

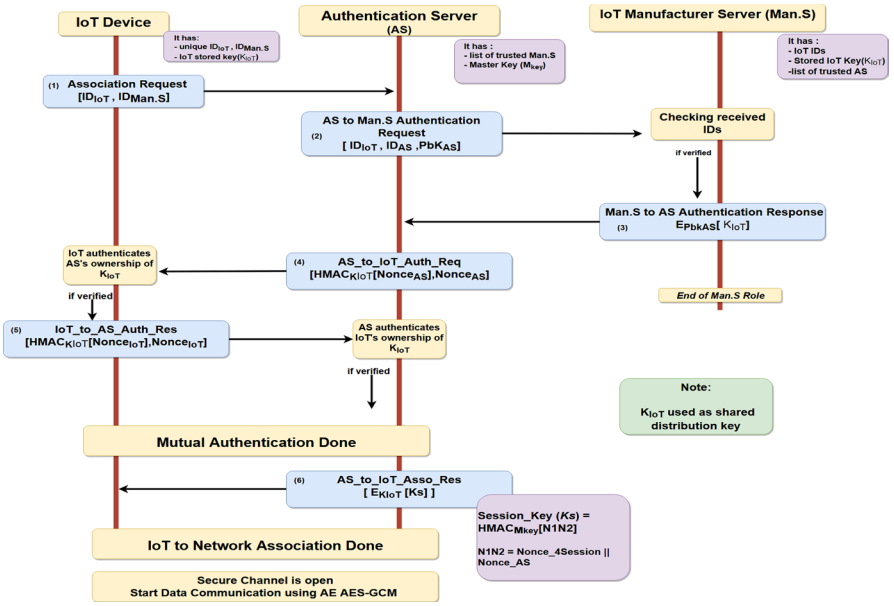


Fig. 9 Proposed IoT device–AS server mutual authentication scheme

received from the AS server in Message 2. At this point, the IoT manufacturer's role in the proposed protocol is completed. This message is illustrated in Fig. 5.

The encryption algorithm used for encrypting K_{IoT} is the RSA-ES [46]. It is asymmetric cryptographic algorithm which can be efficiently computed by both the AS and Man.S servers since they have high processing, storage, and communication capabilities. This is in contrast to the resource-constrained IoT devices which have limited processing, storage, and communication capabilities. Choosing RSA-ES has no affect the system's efficiency, as it is solely used to ensure a secure delivery of K_{IoT}.

3.3.2 The IoT device–AS server communication initialization stage

In this stage, the AS server considers K_{IoT}, which it received securely from IoT manufacturer, as a pre-shared key between itself and the IoT device. Thus, the AS server uses this key to establish a secure communication channel with the IoT device which has the same K_{IoT}, key. This secure channel is deemed necessary to achieve the requested mutual authentication between the AS server and the IoT devices. This is shown in the messages four and five.

Message 4: Authentication request from AS server to IoT device

At this step, the AS server wants to validate the claimed identity of the IoT device which has requested to join the network. To do so, the AS server sends an Authentication Request to the IoT device (*AS to IoT Auth Req*). This request includes a nonce generated at the AS server Nonce_{AS}, in addition to an HMAC [47] generated using

K_{IoT}, which was received from IoT manufacturer in previous message. This message is illustrated in Fig. 6.

Message 5: Authentication response from IoT device to AS server

Upon receiving the *IoT Auth Req* from the AS server, the IoT device computes the HMAC of the received $Nonce_{AS}$, using its pre-shared key K_{IoT}. Then, the device checks the authenticity of the AS server by comparing the two HMACs; the one it received from AS server and the one it computed locally. If the two HMAC values match, then the IoT device authenticates the AS server and it sends back to the server an Authentication Response (*IoT to AS Auth Res*) which includes a nonce $Nonce_{IoT}$ as well the HMAC of the nonce computed using the pre-shared key K_{IoT}. This message is illustrated in Fig. 7.

3.3.3 The exchanged data encryption stage

This stage includes the message 6 which is the final message in this authentication level.

Message 6: Association response from AS server to IoT device

The AS server receives the Authentication Response (*IoT to AS Auth Res*) from the IoT device and computes the HMAC of the received $Nonce_{IoT}$. The server compares the computed HMAC with the sent HMAC, and if a match exists, then this proves the IoT device ownership of the K_{IoT}. Hence, the second authentication factor is verified and the AS server authenticates the IoT device. The two entities are now mutually authenticated and can exchange data. To enable the secure transfer of data between the two entities, the AS server sends as an Association Response (*AS to IoT Assoc Res*) to the IoT device to grant it access to the network. The response message carries a secret session key *Ks*, generated by the AS server and encrypted using the pre-shared key K_{IoT}. This session key will be used by the authenticated encryption algorithm (AE) to encrypt any data transferred between IoT device and the AS server. This message is illustrated as shown in Fig. 8.

4 Performance evaluation of the proposed authentication scheme

In this section, the implementation and evaluation of the proposed IoT device–AS server authentication scheme is presented. The simulation environment which has been used to implement the proposed scheme is described first. This is followed by presenting and analyzing the simulation results. The performance of the scheme is also analyzed with respect to fundamental security requirements. Finally, the scheme is compared with a number of relevant and recent IoT authentication schemes.

4.1 The NS3-based simulation environment

To implement the proposed IoT authentication scheme and evaluate its performance, the NS3 simulator has been used. It is an open-source, discrete-event simulation platform for testing networks, Internet systems, and communication protocols [48,

49]. For this particular research, the NS3 simulator was run on Intel(R) Core(TM) i5 CPUM 460@ 2.53 GHz and 4 GB RAM machine under Ubuntu-16.04.2 platform.

Certain parameters such as device data rate, key size of the crypto-functions, and the version of used crypto-function are changed to study more cases about IoT systems and devices. The ability to simulate the system with different parameters provides in-depth understanding of the effect of each parameter on the overall scheme. Three fundamental parameters are considered; data rate of the IoT device, the hash function on which HMAC is based on, and the bit size of the key which is used for HMAC and for the AES-GCM authenticated encryption function. A combination of different values if these parameters have been simulated and the achieved results have been analyzed with respect to transmission time and communication overheads.

4.1.1 Implementation of network, channel and communicating entities

NS3 supports the simulation of IoT networks through building a customized network architecture which suits the purpose of the simulated scheme. The communicating entities of the IoT system are represented as nodes, and each entity is identified as network device (NetDevice), which is the network layer to the device interface. Figure 10 shows a walkthrough showing how an NS3 project works. Each node has its own specifications, such as its IP address, and its role written in a related application source code. For our proposed scheme, many applications source codes reflecting the functionality of the system have been written. These source codes include the IoT device application source code, the AS server application source code, the IoT manufacture server application source code, the IoT application/user, and the authorization server source code. Each source code is installed on the target node to perform as planned for. The source codes interact and work together through a main program code.

NS3 supports most underlying technologies for IoT devices including 6LoWPAN, IEEE 802.15.4-based LR-WPAN module, IEEE 802.11 Wi-fi module and Internet

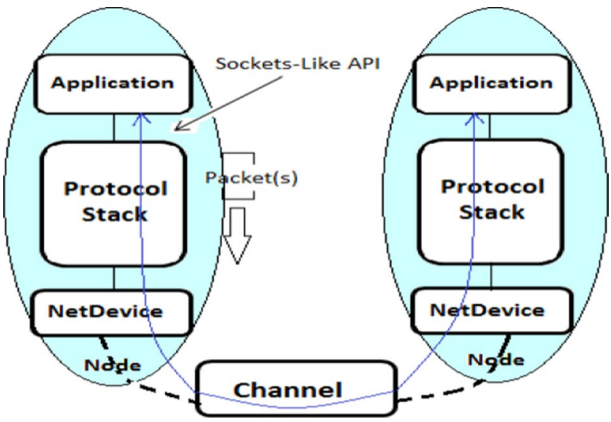


Fig. 10 Workflow of NS3 project [48]

modules IPv4, IPv6, TCP, UDP. The design of the network is also supported by providing modules for network addressing, channel and data rate specifications. It is worthwhile mentioning here that the NS3 processing time is zero time, i.e., the time that NS3 computes is the transmission time, which is a timestamp for each sent and received packet. The specifications of the simulated network is determined in the main code as follows:

- Network topology: point-to-point and star topologies are implemented as shown in Fig. 11.
- Network protocol type: IPv4, UDP, CSMA, Wi-fi, LRWPAN, 6LoWPAN are used.
- Channel data rate: various data rates are used: 5 Mbps, 1 Mbps, and 250 Kbps. The choice of these data rates is based on the data rates specified for the underlying IoT network technologies and according to the specifications of the IoT resource-constrained devices [50].

4.1.2 Implementation of cryptographic functions

Many studies have shown the feasibility of using HMAC functions with IoT devices and systems [21, 23, 24]. Similarly, Authenticated Encryption (AE) functions are lightweight cryptographic functions and have been successfully used in IoT systems due to their security strength, nonce immunity, availability in hardware and software implementation, and speed [41]. One particular AE type is the AES-GCM algorithm which has been found to be lightweight suitable for the resource-constrained IoT devices. AEC-GCM has many attractive features such as a high processing speed, low cost, low latency, parallelization, efficiency, and high performance [38]. Based on these properties, the AES GCM has been a suitable choice to secure IoT data transmission in this proposed scheme. In order to implement the security functions, the cryptographic functions C++ Crypto++ library is used [51].

In what follows, we describe the NS3-based implementation of four procedures: keys generation and derivation procedure, HMAC generation and verification procedure, the AES-GCM Authenticated Encryption algorithm, and the public key cryptography.

4.1.2.1 Keys generation and derivation procedure The keys of the proposed scheme are generated using the combination of the following class functions which Crypto++

```
PointToPointHelper pointToPoint;
pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

NetDeviceContainer p2pDevices;
p2pDevices = pointToPoint.Install (p2pNodes);
```

Fig. 11 Point-to-point network topology used in the proposed system

```
// Now generate the unique saved Key on IoT
          AutoSeededRandomPool prng;
        static SecByteBlock key_i(16);
        prng.GenerateBlock(key_i, key_i.size());
```

Fig. 12 Generation of IoT saved key

```
try
{
      HMAC< SHA1 > hmac(key_IoT, key_IoT.size());

      StringSource(Nonce_AS, true,
            new HashFilter(hmac,
                  new StringSink(HMAC_Nonce)
            ) // HashFilter
      ); // StringSource
}
catch(const CryptoPP::Exception& e)
{
      cerr << e.what() << endl;
      exit(1);
}
```

Fig. 13 HMAC generation

library provides. All the symmetric keys used in this study are generated using these two classes. An example is shown in Fig. 12.

1. *AutoSeededRandomPool Class*. Functions of this class are responsible for generating automatic random seeds pool.
2. *SecBlock Class*. This class provides secure storage or memory block.

The shared session key *Ks* between the IoT device and the AS server are derived using the *Master Key (MKey)* and the HMAC-SHA256 algorithm, as shown in Eqs. 7 and 8.

$$\text{Session key } (Ks) = \text{HMAC}_{\text{Mkey}}(\text{N1N2}) \tag{7}$$

$$\text{N1N2} = \text{Nonce}_{\text{4Session}} || \text{Nonce}_{\text{AS}} \tag{8}$$

where:

- $\text{Nonce}_{\text{4Session}}$ is a nonce AS server generates specifically for generating the session key Ks.
- Nonce_{AS} is the same nonce that AS uses in the *AS to IoT Auth Req*.

4.1.2.2 HMAC generation and verification The AS server computes the HMAC to create its nonce (Nonce_{AS}) using the K_{IoT}. An HMAC_nonce as generated shown in Eq. (9). HMAC generation and verification code is partially shown in Fig. 13.

$$\mathrm{HMAC}_{\mathrm{nonce}} = \mathrm{HMAC}\left(\mathrm{Nonce}_{\mathrm{AS}}, K_{\mathrm{IoT}}\right) \tag{9}$$

4.1.2.3 Implementation of authenticated encryption AES GCM algorithm Crypto++ provides an interface that facilitates the implementation of the AES GCM algorithm by using the GCM object with two filters: *AuthenticatedEncryptionFilter* and *AuthenticationDecryptionFilter*. The following parameters have to be set for these filters to be used by the sender and receiver:

- The secret key and its size.
- An Initialization Vector (IV) and its size.
- Tag size.

Figure 14 shows a part of the algorithm code for constructing and using an *AuthenticatedEncryptionFilter.* This filter will throw an exception of any of its input is not valid.

4.1.2.4 Implementation of the public key cryptographic functions The computation-intensive public key cryptography is used by the AS the Man.S servers which have rich computational resources. The RSA algorithm has been used for encryption the of K_{IoT} key sent from the Man.S server to the AS server. Figure 15 shows how this algorithm is implemented using the crypto++.

4.2 Simulation results

IoT systems are time-sensitive, and thus, achieving real-time efficiency of any proposed authentication scheme is a major objective. In other words, fast response and fast data transmission are deemed necessary for effective mutual authentication schemes. The simulation results are presented here in order to evaluate the time efficiency and communication overhead of the proposed scheme. A trade-off between

```
try
{
GCM< AES >::Encryption e;
e.SetKeyWithIV(key_IoT, key_IoT.size(), Iv, 16);
//e.SetKeyWithIV(Session_Key, Session_Key.size(), Iv, 16);

      StringSource(session_Key, true,
            new AuthenticatedEncryptionFilter(e,
                  new StringSink(ciphered_Sk)
            ) // StreamTransformationFilter
      ); // StringSource
}
catch(const CryptoPP::Exception& e)
{
      cerr << e.what() << endl;
      exit(1);
}
```

Fig. 14 Implementation of AES GCM encryption

```
// encrypte key using AS server public key
generated at AS and sent to Man.S server
 RSA::PublicKey Pbk;
 string ciphered_iot_k;
 //Encryption RSAES
  RSAES_OAEP_SHA_Encryptor e(Pbk);
  StringSource ss1(S_key_iot, true,
 new PK_EncryptorFilter(prng, e,
     new StringSink(ciphered_iot_k)
  ) // PK_EncryptorFilter
  ); // StringSource
```

Fig. 15 RSA encryption

security level, time efficiency, and communication overhead is analyzed. This is done for different data rates and different security settings.

4.2.1 Time efficiency

To evaluate the time efficiency of the proposed scheme, it is necessary to measure the time spent in the transmission of packets between the communicating entities. In NS3, the time can be measured using events time in the NS3 terminal. Figure 16 shows the results produced by NS3 when the IoT device and the AS server execute

```
islam@ubuntu: ~/Downloads/ns-allinone-3.28/ns-3.28
At time 2s IoT client sent Association Request of  14 bytes to AS Authentication
 Server with IP Address 10.1.1.2 port 9
At time 2.00207s AS Authentication Server received 14 bytes from 10.1.1.1 port 4
9153
Received IoT ID 19A0001AReceived Manu_ID AP285
SUCCESS TO CONNECT TO MANUFACTURER SERVER
At time 2.00207s AS Authentication Server sent 177 bytes to MANUFACTURER SERVER
with IP 10.1.2.2 port 49153
AS server_ID Ed734512
 IoT_ID 19A0001A
Ckecking status (validity) of received IDs ...
 AS server is Trusted
IoT ID is Valid
Send IoT saved key to AS Authentication Server
At time 2.00666s AS Authentication Server received 129 bytes from 10.1.2.2 port
49154
Nonce_AS  437199346
HMAC_Nonce 32
 Send Authentication Request to IoT  57
SUCCESS TO CONNECT TO IoT
At time 2.0088s IoT client received  58 bytes from 10.1.1.2 port 9
Authentication Request from AS server Received  57
received_AS_Nonce is of size  21
received_iv is of size  16
Verified AS Authentication Server
Nonce_IoT  1804289383
 Send Authentication Response to AS  42
At time 2.01091s AS Authentication Server received 43 bytes from 10.1.1.1 port 4
9153
Authentication Response from IoT Received  42
received_iot_Nonce is of size  22
Verified IoT device .... IoT<--> AS AUTHENTICATION SERVER MUTUAL AUTHENTICATION
VERIFIED
Ssion_Key  32
At time 2.01304s IoT client received  49 bytes from 10.1.1.2 port 9
recovered_Sk : 32
```

Fig. 16 Time measurement for the proposed scheme using NS3 events log terminal

Table 2 Time measurements for the proposed scheme at data rate of 5Mbps and 32 bytes key size for HMAC-SHA256

Message no	Message size/bytes	Sender Id	Sending time/s	Receiver Id	Receiving time/s
1	14	IoT device	2.00000	AS server	2.00207
2	177	AS server	2.00207	Man.S	2.00415
3	129	Man.S	2.00415	AS server	2.00666
4	58	AS server	2.00666	IoT device	2.0088
5	43	IoT device	2.0088	AS server	2.01091
6	49	AS server	2.01091	IoT device	2.01304

Table 3 Communication overhead for the proposed scheme at data rate of 5 Mbps and 32 bytes key size for HMAC-SHA256

$e(n)$	Msg_i/bytes	$C_{e(n)}$/bytes
IoT device	Msg_1 (14 bytes)	57 bytes
	Msg_5 (43 bytes)	
AS server	Msg_2 (177 bytes)	284 bytes
	Msg_4 (58 bytes)	
	Msg_6 (49 bytes)	
Man.S	Msg_3 (129 bytes)	129 bytes
		$C_a = 470$

the mutual authentication process for 5 Mbps data rate, 32 bytes key size, and HMAC-SHA256 hashing. The time is shown at the main events, specifically, at the transmission and receiving events. Since the NS3 processing time is zero time, the transmission time which NS3 computes is based on the timestamp for each sent and received packet. Table 2 shows this timestamps for the six messages exchanged in the mutual authentication process.

Referring to Table 2, assuming that *Ts* refers to time of sending the first message, i.e., message no.1, *Tr:* refers to the time of receiving the final message, i.e., message no.6, then *Tt* which is the total time spent by the six messages of the authentication process can be computed as follows:

$$\begin{aligned} Tt &= Tr - Ts \\ &= 2.01304 - 2.00000 \\ &= 0.01304\,\text{s or } 13.04\,\text{ms} \end{aligned} \tag{10}$$

That is, the proposed scheme spends a total time of 13.04 ms for mutual authentication process, including session key exchange at 5 Mbps data rate. It also measures the network latency, indicating that the time delays in the proposed scheme are small, thus meeting the time efficiency requirement of IoT systems and networks.

4.2.2 Communication overhead

The communication overhead is measured by the number of bytes in every communication message sent. Table 3 gives the communication overhead of the entities of the proposed scheme during the mutual authentication process. The communication overhead calculation is based on Eqs. (11) and (12).

$$C_{e(n)} = \sum \mathrm{Msg}_i \tag{11}$$

$$C_a = \sum C_{e(n)} \tag{12}$$

where $C_{e(n)}$ refers to the communication overhead of a specific entity, $e(n)$ refers to one of three entities as follows: {$e(1)$: IoT device, $e(2)$: AS server, $e(3)$: Man.S}, C_a refers to the total communication overhead of all entities, Msg_i refers to size of the message sent by an entity, and i indicates the number of the exchanged message: $i = \{1, 2, 3, 4, 5, 6\}$.

4.2.3 Computational overhead

The computational overhead is measured by the time and the resources needed to process the packets or perform validation operations on the received packets [52]. NS3 has zero processing (computation) time, so no computational overhead is measured. However, using symmetric encryption algorithms in the context of IoT systems is more efficient and applicable rather than using asymmetric algorithms with regard to the amount of computational overhead [21]. The later needs high processing capabilities and requires long keys to provide a certain level of security. The resource-constrained IoT devices are not capable to process and handle the requirements of asymmetric cryptography which include large random numbers, large public and private keys, and long encryption and decryption operations [24, 35]. Generally speaking, the computational overhead of the proposed scheme is low since the scheme achieves mutual authentication using a limited number of small-sized messages, lightweight cryptographic functions, and passive discovery of IoT device. Moreover, most of the power/energy of the IoT devices is consumed in the communication process, whether in transmitting or receiving data [53].

4.2.4 Effect of data rates and type of Hah functions on time efficiency

Referring to Fig. 17, it can be noticed that the data rate has a significant and direct effect on the time efficiency, and within the same data rate, the time increases slightly with the use of higher versions of the SHA algorithm. As shown in the figure, increasing the data rate causes a noticeable increase in time efficiency. For example, using a date rate of 250 Kbps and SHA512, the recorded time is 34.865 ms. This is almost a two and half times increase over the time recorded for the 5 Mbps data rate and SHA512, which is 13.14 ms. However, the difference in the time recorded for the 1Mbps and 5Mbps data rates is modest. Similarly, Fig. 18

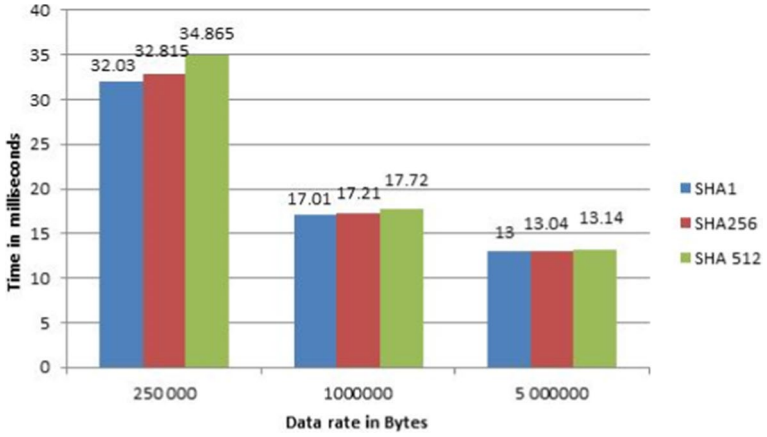


Fig. 17 Time efficiency comparison between different hashing functions

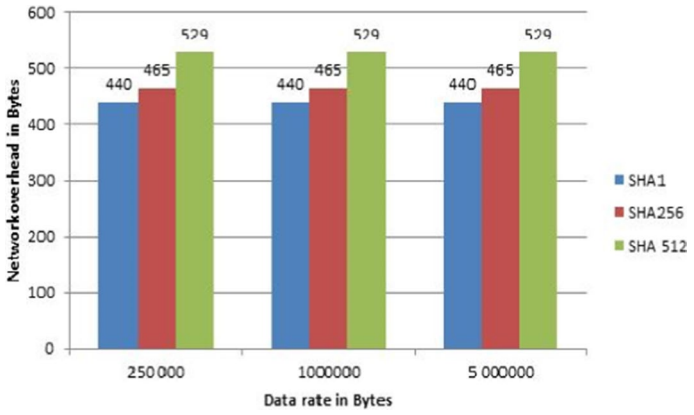


Fig. 18 Communication overhead comparison for different hashing functions

demonstrates the performance of the proposed scheme with regard to the communication overhead which is expressed by the total number of bytes. It can been noticed from the figure that communication overhead is not affected by the data rate. However, it increases when using higher SHA versions due to the increase in the size of the generated HMAC.

4.3 Security analysis

The proposed scheme achieves the essential security requirements and provides robustness against common attacks. This achievement has been provided by the scheme by virtue of a number of implementations issues carried out by

the scheme. The first issue is the establishment of a secure channel between the IoT devices and the AS server by means of a session key agreement. The second issue is the employment of a secure HMAC, pre-stored IoT key, and the use of nonce tokens to enable the communicating entities to verify each other. The third issue is using the lightweight AES-GCM standard to protect the data exchanged between the entities. In what follows, we show how the scheme achieves the essential security requirements and how it counters common cyber-attacks.

4.3.1 Achieving essential security requirements

The essential security requirements which have been met by the proposed IoT authentication scheme includes two-factor mutual authentication, data confidentiality, integrity, and authenticity. A description of how each security requirement has been achieved is given as follows.

1. *Mutual authentication*. Achieved through provisioning a piece of secret information possessed by communicating parties only. This information is the ID of the IoT device itself, and the pre-stored IoT key K_{IoT}.
2. *Confidentiality*. Achieved by minimizing number of plaintext messages, and by ensuring that no usable information is readable by unauthorized eavesdropper. This is done by encrypting and hashing the useful information and exchanged secrets by messages 3 to 6.
3. *Integrity*. Achieved by using the keyed hash function (HMAC) of the exchanged nonce to ensure the identity of the other party through having K_{IoT}; the secret key of the IoT device.
4. *Authenticity*. Achieved using the authenticated encryption algorithm AES-GCM to encrypt the session keys and any data transfers.
5. *Availability*. Achieved by configuring a backup AS server, and by implementing passive discovery and self-registration of IoT devices. The self-registration protects the IoT device from DoS and DDoS attacks.
6. *Privacy and access control.* Achieved as a byproduct of preserving the confidentiality, integrity, and authenticity.

4.3.2 Robustness against common cyber-attacks

The proposed scheme exhibits robustness against some of the well-known cyber-attacks such as the impersonation attacks, replay attacks, spoofing, sniffing, and man-in-the-middle-attacks, session key brute force attack, denial-of-service attacks, and identity theft. These attacks are naturally prevented by the six messages of the proposed scheme. In Table 4, the attacks and the messages responsible for preventing each attack are listed.

Table 4 Scheme's robustness against common cyber-attacks

The cyber-attack	Prevention mechanism	Reference message(s)
Impersonation attack	Prevented by the entire mutual authentication process between the IoT devices and AS server	Messages No. 1–6
Replay attack	Prevented by using nonce and sessions keys in the mutual authentication process	Messages No. 4,5
Spoofing, sniffing, and man-in-the-middle-attacks	Prevented by avoiding sending secret data or keys in plaintext. Data and secret keys encrypted then sent	Messages No. 3–6
Denial-of-Service (DoS) attacks	Prevented by authenticating the IoT devices before granting them access to the network	Messages No. 2–5
Identity theft	Prevented through using two-factor authentication; the IoT ID itself and the ownership of the IoT saved key	Messages No. 2–5
Session key brute force attack	Prevented by avoiding the exchange of parameters used in generating session key Ks (i.e., nonce4session and master key). So, no attacker can generate the secret session key	Message No.6

Table 5 Security and efficiency comparison between the proposed scheme and related works

Performance metric	[30]	[31]	[32]	[33]	[34]	[35]	[36]	[37]	Proposed scheme
Number of exchanged messages	8	11	7	6	9	8	3	8	6
Security of exchanged messages	O	X	O	X	–	O	–	O	O
Efficient utilization of IoT device resources	X	X	X	–	X	X	X	X	O
Automated key distribution system scalability (no human intervention)	O	–	X	X	–	X	O	–	O
Use of lightweight cryptography	X	O	O	O	O	O	X	X	O
Passive discovery of IoT devices	–	X	–	O	X	–	X	–	O
IoT identity validation	O	X	–	X	X	X	O	O	O
Establishment of a secure channel	O	O	X	O	–	O	–	O	O
Saving IoT time for non-security functions	X	X	X	–	X	X	X	X	O
Availability	–	X	X	–	X	X	X	X	O
Communication Overhead (in bytes)	1639 bytes	–	–	–	–	–	–	–	470 bytes

O, achieved; *X*, not achieved; –, not available (N/A)

4.4 Comparison with relevant IoT authentication schemes

To demonstrate the effectiveness of the proposed scheme a security and performance comparison with related previous schemes is shown in Table 5. Comparison has been made with respect to the following performance parameters: number of exchanged messages, security of exchanged messages, efficient utilization of IoT device resources, automated key distribution/system scalability/no human intervention, use of lightweight cryptography, passive discovery of IoT devices, IoT identity validation, establishment of a secure channel, saving IoT time, availability, and communication overhead. Referring to the table, it can be concluded that the proposed mutual authentication IoT scheme meets all requirements deemed necessary for efficient mutual authentication of IoT systems. Moreover, the scheme outperforms most of the schemes it has been compared with.

5 Conclusion and future research directions

For any IoT authentication scheme to be considered applicable and suitable for IoT systems, it must fulfill a number of requirements which have been based on the characteristics and limitations of IoT devices. These requirements include using lightweight cryptographic functions, adopting standard communication protocols,

providing real-time efficiency, ensuring low communication overhead, allowing minimal human intervention, providing dynamicity and scalability to support automated IoT device association.

In this paper, we presented an effective IoT mutual authentication scheme based an innovative trust model between IoT devices, IoT manufacturer, and a middleware. The model aims to establish a practical, automated and efficient two-factor mutual authentication scheme that can be used to build a fully functional and secure IoT network. This mutual authentication scheme is based on the identified parties and structure of the IoT systems, taking into account the most common design aspects, data flows, hardware constraints, communicational and computational overheads, time efficiency, and security requirements. The scheme makes use of the lightweight cryptographic functions and HMACs. The proposed scheme uses the standard User Datagram Protocol (UDP), which shows less overhead in terms of energy and battery power consumption compared to the TCP protocol. The UDP is an unreliable protocol; however, it is more suitable for real-time IoT applications where time is critical.

The mutual authentication process between the different IoT entities was accomplished by the exchange of only six small messages in three successive stages; the IoT device association stage, the IoT device–AS server communication initialization stage, and the exchanged data encryption stage. This six-message scheme was simulated using the Network Simulator NS3 with C++ crypto++ library, using various IoT enabling technology models like such as Wi-fi, 6LoWPAN, and LR-WPAN. Simulation results verified the proper functioning of the scheme and demonstrated its efficiency regarding computational and communication overheads. The scheme needed 13.04 ms and 470 bytes to complete the mutual authentication process, including the session key exchange process.

The proposed scheme was analyzed for security, and showed robustness against common cyber-attacks. However, there remains the need to launch actual attacks to verify robustness. This is a major limitation that will be handled as a future research issue. Nevertheless, and as a conclusion, we believe that the IoT authentication scheme proposed in this paper has met all requirements deemed necessary for efficient mutual authentication of IoT systems. That is, the scheme uses lightweight cryptographic algorithms like AES and HMAC. It also requires a small number of exchanged messages and bytes to ensure minimal network and communication overheads, thus coping with the resource-constrained nature of IoT devices and networks. The system's dynamicity and scalability are ensured by allowing limited human intervention for the installation of the pre-shared keys in the IoT devices.

The ongoing and future research goes into three directions. In the first direction, the current scheme will be further evaluated in terms of its robustness against a whole set of actual cyber-attacks. A proper simulation environment will be set to launch the selected attacks. The second direction involves extending the proposed mutual authentication scheme to provide authentication between the IoT devices themselves. Another higher level of extension is to devise an end-to-end authentication scheme that will provide authentication between an IoT device and an IoT application/user. In the third and last direction, the authors will investigate the possibility of using the Blockchain technology to solve some pending IoT scalability

and security issues. The Blockchain characteristics such as the heavy cryptography and the large storage requirements may not readily suit the resource-constrained IoT devices [54–60]; however, extensive research efforts are required in this direction.

Funding This research was not funded.

Declarations

Conflict of interests This article has no potential conflicts of interest.

Research involving human participants and/or animals This article does not contain any studies with human participants or animals performed by any of the authors.

Informed consent N/A.

References

1. Samaila MG, Neto M, Fernandes DA, Freire MM, Inácio PR (2017) Security challenges of the Internet of Things. In: Batalla J, Mastorakis G, Mavromoustakis C, Pallis E (eds) Beyond the Internet of Things. Springer, Cham, pp 53–82
2. Cisco, Inc (2016) Internet of Things at a glance Retrieved 19 May 2019 from https://www.cisco.com/c/dam/en/us/products/collateral/se/internet-of-things/at-a-glance-c45-731471.pdf
3. Hammoudi S, Aliouat Z, Harous S (2018) Challenges and research directions for Internet of Things. Telecommun Syst 67(2):367–385
4. Kumari S, Karuppiah M, Das AK et al (2018) A secure authentication scheme based on elliptic curve cryptography for IoT and cloud servers. J Supercomput 74:6428–6453. https://doi.org/10.1007/s11227-017-2048-0
5. Kim H (2017) Securing the Internet of Things via locally centralized, globally distributed authentication and authorization. Doctoral dissertation, UC Berkeley
6. Sudha MN, Rajendiran M, Specht M et al (2021) A low-area design of two-factor authentication using DIES and SBI for IoT security. J Supercomput. https://doi.org/10.1007/s11227-021-04022-w
7. Oh J, Yu S, Lee J, Son S, Kim M, Park Y (2021) A secure and lightweight authentication protocol for IoT-based smart homes. Sensors 21:1–24
8. Adat V, Gupta BB (2018) Security in Internet of Things: issues, challenges, taxonomy, and architecture. Telecommun Syst 67(3):423–441
9. Sadhukhan D, Ray S, Biswas GP et al (2021) A lightweight remote user authentication scheme for IoT communication using elliptic curve cryptography. J Supercomput 77:11141151. https://doi.org/10.1007/s11227-020-03318-7
10. Lakhan A, Mohammed MA, Obaid OI et al (2022) Efficient deep-reinforcement learning aware resource allocation in SDN-enabled fog paradigm. Autom Softw Eng 29:20. https://doi.org/10.1007/s10515-021-00318-6
11. Lakhan A, Mohammed MA, Kadry S, Abdulkareem KH, Al-Dhief FT, Hsu CH (2021) Federated learning enables intelligent reflecting surface in fog-cloud enabled cellular network. PeerJ Comput Sci 7:e758. https://doi.org/10.7717/peerj-cs.758
12. Lakhan A, Memon MS, Mastoi Q et al (2021) Cost-efficient mobility offloading and task scheduling for microservices IoVT applications in container-based fog cloud network. Cluster Comput. https://doi.org/10.1007/s10586-021-03333-0
13. Choi S, Ko J, Kwak J (2019) A study on IoT device authentication protocol for high speed and lightweight. In: International Conference on Platform Technology and Service (PlatCon) 2019, pp 1–5.https://doi.org/10.1109/PlatCon.2019.8669418
14. Chuang M-C, Yen C-C (2021) Geometric authentication mechanism for enhancing security in IoT environment. Symmetry 13(8):1369

15. Fremantle P, Scott P (2017) A survey of secure middleware for the Internet of Things. PeerJ Comput Sci 3:e114
16. Tewari A, Gupta BB (2017) Cryptanalysis of a novel ultra-lightweight mutual authentication protocol for IoT devices using RFID tags. J Supercomput 73:1085–1102. https://doi.org/10.1007/s11227-016-1849-x
17. Fakroon M, Alshahrani M, Gebali F, Traore I (2020) Secure remote anonymous user authentication scheme for smart home environment. Internet Things 9:100158
18. Lin J, Yu W, Zhang N, Yang X, Zhang H, Zhao W (2017) A survey on Internet of Things: architecture, enabling technologies, security and privacy, and applications. IEEE Internet Things J 4(5):1125–1142
19. Kang B, Han Y, Qian K, Jianqi Du (2020) Analysis and improvement on an authentication protocol for IoT-enabled devices in distributed cloud computing environment. Math Probl Eng. https://doi.org/10.1155/2020/1970798
20. Huang H, Lu S, Wu Z et al (2021) An efficient authentication and key agreement protocol for IoT-enabled devices in distributed cloud computing architecture. J Wirel Commun Netw 2021:150. https://doi.org/10.1186/s13638-021-02022-1
21. Pereira GC, Alves RC, Silva FLD, Azevedo RM, Albertini BC, Margi CB (2017) Performance evaluation of cryptographic algorithms over IoT platforms and operating systems. Secur Commun Netw 2017:1–16
22. Toshihiko O (2017) Lightweight cryptography applicable to various IoT devices. NEC Tech J 12(1):67–71
23. Kiran SKVVNL, Harini N (2018) Evaluating efficiency of HMAC and digital signatures to enhance security in IoT. Int J Pure Appl Math 119:13991–13997
24. Sfar AR, Natalizio E, Challal Y, Chtourou Z (2018) A roadmap for security challenges in the Internet of Things. Digit Commun Netw 4(2):118–137
25. Xue K, Meng W, Li S, Wei DSL, Zhou H, Yu N (2019) A secure and efficient access and handover authentication protocol for Internet of Things in space information networks. IEEE Internet Things J 6(3):5485–5499. https://doi.org/10.1109/JIOT.2019.2902907
26. El-hajj M, Fadlallah A, Chamoun M, Serhrouchni A (2019) A survey of Internet of Things (IoT) authentication schemes. Sensors 19(5):1141
27. Melki R, Noura HN, Chehab A (2020) Lightweight multi-factor mutual authentication protocol for IoT devices. Int J Inf Secur 19:679–694. https://doi.org/10.1007/s10207-019-00484-5
28. Sanchez-Gomez J, Garcia-Carrillo D, Marin-Perez R, Skarmeta AF (2020) Secure authentication and credential establishment in narrowband IoT and 5G. Sensors (Basel) 20(3):882. https://doi.org/10.3390/s20030882
29. Ferrag MA, Maglaras LA, Janicke H, Jiang J, Shu L (2017) Authentication protocols for Internet of Things: a comprehensive survey. Secur Commun Netw 2017:1–41
30. Goworko M, Wytrębowicz J (2021) A secure communication system for constrained IoT devices—experiences and recommendations. Sensors 21:6906. https://doi.org/10.3390/s21206906
31. Liu X, Zhao M, Li S, Zhang F, Trappe W (2017) A security framework for the Internet of Things in the future internet architecture. Future Internet 9(3):27
32. Park N, Kang N (2016) Mutual authentication scheme in secure Internet of Things technology for comfortable lifestyle. Sensors 16(1):20
33. Hammi MT, Livolant E, Bellot P, Serhrouchni A, Minet P (2017, October) A lightweight IoT security protocol. In: 2017 1st Cyber Security in Networking Conference (CSNet). IEEE, pp 1–8
34. Bala DQ, Maity S, Jena SK (2017, May) Mutual authentication for IoT smart environment using certificate-less public key cryptography. In: 2017 Third International Conference on Sensing, Signal Processing and Security (ICSSS). IEEE, pp 29–34
35. Rabiah AB, Ramakrishnan KK, Liri E, Kar K (2018, February) A lightweight authentication and key exchange protocol for IoT. In: Workshop on Decentralized IoT Security and Standards (DISS).
36. Challa S, Wazid M, Das AK, Kumar N, Reddy AG, Yoon EJ, Yoo KY (2017) Secure signature-based authenticated key establishment scheme for future IoT applications. IEEE Access 5:3028–3043
37. Kothmayr T, Schmitt C, Hu W, Brünig M, Carle G (2013) DTLS based security and two-way authentication for the Internet of Things. Ad Hoc Netw 11(8):2710–2723
38. Edgar TW, Manz DO (2017) Research methods for cyber security. Syngress, Oxford
39. Gueron S (2013) AES-GCM for efficient authenticated encryption—ending the reign of HMAC-SHA-1. Real-World Cryptography. Retrieved 21 May 2019 from https://crypto.stanford.edu/RealWorldCrypto/slides/gueron.pdf

40. Al-Haj A, Abandah G, Hussein N (2015) Crypto-based algorithms for secured medical image transmission. IET Inf Secur 9(6):365–373
41. Koteshwara S, Das A (2017) Comparative study of Authenticated Encryption targeting lightweight IoT applications. IEEE Design Test 34(4):26–33
42. Aghili SF, Ashouri-Talouki M, Mala HD (2018) impersonation and de-synchronization attacks against an ultra-lightweight RFID mutual authentication protocol for IoT. J Supercomput 74:509–525. https://doi.org/10.1007/s11227-017-2139-y
43. Sadhukhan D, Ray S, Biswas GP, Khan MK, Dasgupta M (2021) A lightweight remote user authentication scheme for IoT communication using elliptic curve cryptography. J Supercomput 77(2):1114–1151
44. Shuai M, Yu N, Wang H, Xiong L (2019) Anonymous authentication scheme for smart home environment with provable security. Comput Secur 86:132–146
45. Dworkin M, Barker E, Nechvatal J, Foti J, Bassham L, Roback E, Dray J (2001) Advanced Encryption Standard (AES), Federal Information Processing Standards (NIST FIPS), National Institute of Standards and Technology, Gaithersburg, MD [online]. https://doi.org/10.6028/NIST.FIPS.197. Accessed 22 Feb 2022
46. Rivest RL, Shamir A, Adleman LM (1978) A method for obtaining digital signature and public-key cryptosystems. Commun ACM 21(2):120–126
47. Bellare M, Canetti R, Krawczyk H (1996) Keying hash functions for message authentication. In: Koblitz N (ed) Advances in Cryptology, Proceedings Crypto '96, LNCS 1109. Springer-Verlag, Berlin, pp 1–15
48. NSNAM. Network Simulator 3—NS3. (Online). https://www.nsnam.org/. Accessed 22 Feb 2022
49. NS-3-Tutorial (2018) ns-3 tutorial, release ns-3.29. (Online). https://www.nsnam.org/docs/release/3.29/tutorial/ns-3-tutorial.pdf. Accessed 22 Feb 2022
50. Metongnon L, Ezin EC, Sadre R (2017, May) Efficient probing of heterogeneous IoT networks. In: 2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM). IEEE, pp 1052–1058
51. Crypto++ (2022). (Online). https://cryptopp.com/. Accessed 22 Feb 2022
52. Nabil M, Bima M, Alsharif A, Johnson W, Gunukula S, Mahmoud M, Abdallah M (2019) Priority-based and privacy-preserving electric vehicle dynamic charging system with divisible e-payment. In: Rawat DB, Ghafoor KZ (eds) Smart cities cybersecurity and privacy. Elsevier, Amsterdam. https://doi.org/10.1016/B978-0-12-815032-0.00012-3
53. Schmitt C, Kothmayr T, Hu W, Stiller B (2017) Two-way authentication for the internet-of-things. In: Acharjya D, Geetha M (eds) Internet of things: novel advances and envisioned applications. Springer, Cham, pp 27–56
54. Christidis K, Devetsikiotis M (2016) Blockchains and smart contracts for the Internet of Things. IEEE Access 4:2292–2303
55. Dorri A, Kanhere SS, Jurdak R, Gauravaram P (2017) LSB: a lightweight scalable blockchain for IoT security and privacy. arXiv preprint, arXiv:1712.02969
56. Khan MA, Salah K (2018) IoT security: review, blockchain solutions, and open challenges. Futur Gener Comput Syst 82:395–411
57. Wu L, Du X, Wang W, Lin B (2018, March). An out-of-band authentication scheme for Internet of Things using blockchain technology. In: 2018 International Conference on Computing, Networking and Communications (ICNC). IEEE, pp 769–773
58. lakhan A, Mohammed MA, Ibrahim DA, Abdulkareem KH (2021) Bio-inspired robotics enabled schemes in blockchain-fog-cloud assisted IoMT environment. J King Saud Univ – Comput Inf Sci. https://doi.org/10.1016/j.jksuci.2021.11.009
59. Lakhan A, Mohammed MA, Rashid AN, Kadry S, Panityakul T, Abdulkareem KH, Thinnukool O (2021) Smart-contract aware ethereum and client-fog-cloud healthcare system. Sensors 21:4093. https://doi.org/10.3390/s21124093
60. Lakhan A, Mohammed MA, Kozlov S, Rodrigues JJPC (2021) Mobile-fog-cloud assisted deep reinforcement learning and blockchain-enable IoMT system for healthcare workflows. Trans Emerg Telecommun Technol. https://doi.org/10.1002/ett.4363

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Islam Alshawish[1] · Ali Al-Haj[1]

Islam Alshawish
ialshawish@gmail.com

[1] Department of Computer Engineering, The King Abdullah II School of Engineering, Princess Sumaya University for Technology, Al-Jubeiha, P.O. BOX 1438, Amman 11941, Jordan